

Module VII — Programming Syntax & Logic

How Modern Programmers Think

Cloud Contraptions LLC - www.cloudcontraptions.com

Overview

What This Module Covers

- From concepts to real, running code
- Anatomy of a programming language
- JavaScript fundamentals and hands-on VS Code
- Modern web development overview
- Introduction to Java
- Two languages, one set of core ideas

Learning Objectives

- Distinguish syntax (grammar) from semantics (meaning)
- Compare compiled, interpreted, and JIT languages
- Write JavaScript variables, functions, and control flow
- Use VS Code to run and debug code
- See how the web stack fits together
- Move from JavaScript to Java

Anatomy Of A Programming Language

Syntax, Semantics, And Execution

- Every language defines three essentials
- Syntax — grammar rules for writing code
- Semantics — what the code actually means
- Execution model — how the computer runs it
- Syntax errors stop code; semantic errors mislead

```
let x = "hello" + 5;    // valid syntax, becomes "hello5"
```

Compilation vs. Interpretation

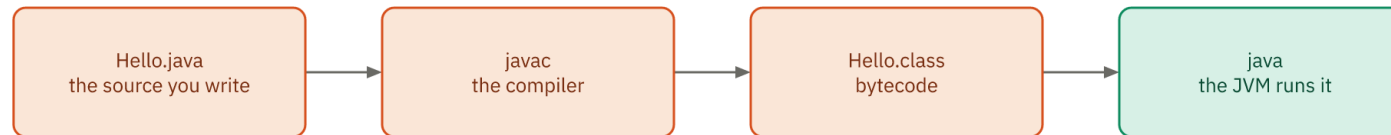
- Compiled: fully translated before running
- Interpreted: read and run line by line
- Compiled runs faster and catches errors early
- Interpreted gives instant feedback and portability
- JIT blends both (Java, JavaScript engines)

Compiled and Interpreted, Drawn

Compiled and Interpreted

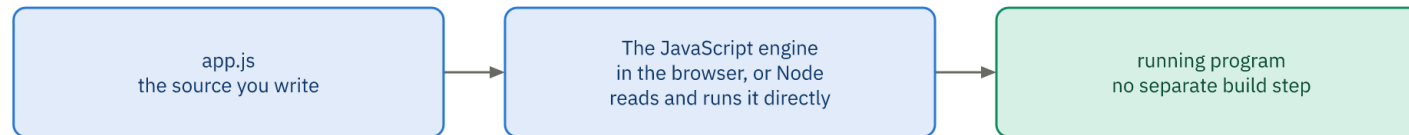
The difference is when your mistakes are found.

COMPILED — Java



A typo or a type mistake stops you HERE — before the program has run, and before any user sees it.

INTERPRETED — JavaScript



The same mistake is not found until the line actually runs — which may be in front of a user.

Compiling buys safety and costs a step. Interpreting buys speed of trying things and costs late surprises. Neither is the winner.

Typing: Static, Dynamic, Strong, Weak

Axis	One End	Other End
Declaration	Static — Java	Dynamic — JS
Enforcement	Strong — Java	Weak — JS

- Static: types declared and checked early
- Dynamic: types inferred while running
- Weak typing coerces: `"5" + 3` becomes `"53"`

The Building Blocks Of A Program

- Keywords: reserved words like `let`, `if`, `return`
- Statements do something; expressions produce values
- Operators: arithmetic, comparison, logical, assignment
- Variables are named containers for values
- Comments explain "why," not "what"

Three Kinds Of Errors

- Syntax errors: broken grammar; won't run
- Runtime errors: valid code fails mid-run
- Logic errors: runs fine, wrong result
- Logic errors are the hardest to find
- Testing and reasoning catch logic errors

JavaScript Fundamentals

Why JavaScript First

- Only language running natively in browsers
- Immediate feedback; results in seconds
- Runs in browser, Node.js, desktop, mobile
- Forgiving and flexible for beginners
- Core concepts transfer to every language

Variables: `const`, `let`, `var`

- `const` by default — cannot be reassigned
- `let` when the value will change
- Avoid `var` — old, function-scoped quirks
- `const` blocks reassignment, not mutation

```
const pi = 3.14159;  
let count = 0;  
count = count + 1;    // now 1
```

Core Data Types

- string, number, and boolean values
- `null` is intentional; `undefined` means unset
- object holds key-value pairs; array is ordered
- Use `===` and `!==` for safe comparison
- `typeof` reveals a value's type

```
let name = "Alice";    // string
let scores = [90, 85]; // array
```

Conditionals And Loops

- `if / else if / else` makes decisions
- Ternary: `cond ? a : b` shorthand
- `for` and `while` repeat work
- `for...of` iterates over array values

```
for (let i = 0; i < 3; i++) {  
  console.log(i);    // 0, 1, 2  
}
```


Functions

- Reusable, named blocks of logic
- Parameters are inputs; `return` sends a value back
- Declarations, expressions, and arrow functions
- Arrow functions give concise syntax

```
function add(a, b) {  
  return a + b;  
}  
const double = n => n * 2;
```

Arrays: map, filter, reduce

- `push` and `pop` add/remove at the end
- `map` transforms every element
- `filter` keeps elements matching a test
- `reduce` combines all into one value

```
let nums = [1, 2, 3, 4, 5];  
let evens = nums.filter(n => n % 2 === 0);  
let sum = nums.reduce((acc, n) => acc + n, 0);
```

Objects

- Collections of key-value pairs
- Access with dot or bracket notation
- Methods are functions inside objects
- `this` refers to the object itself

```
let person = {  
  name: "Alice",  
  greet() { return "Hi, " + this.name; }  
};  
person.greet();    // "Hi, Alice"
```

Hands-On Practice In VS Code

Setting Up VS Code

- Free, lightweight, works with any language
- Download from code.visualstudio.com
- Add extensions: ESLint, Prettier, Live Server
- Install Node.js to run JavaScript locally
- Write, run, see results — then repeat

Running JavaScript With Node.js

- Node.js runs JavaScript outside the browser
- Open VS Code's integrated terminal
- Run a file with `node filename.js`
- Predict the output, then compare to reality

```
// hello.js  
console.log("Hello from VS Code!");  
// terminal: node hello.js
```

Running JavaScript In The Browser

- HTML page loads your JavaScript file
- JavaScript reacts to clicks and input
- "Open with Live Server" to preview
- Browser auto-refreshes as you edit

```
let button = document.getElementById("go");
button.addEventListener("click", function() {
    document.getElementById("msg").textContent = "Clicked!";
});
```

Reading Error Messages

- Error messages point to what went wrong
- `SyntaxError`: broken grammar, won't parse
- `ReferenceError`: variable does not exist
- `TypeError`: invalid operation on a value
- Read the line number, then reason

Modern Web Development Overview

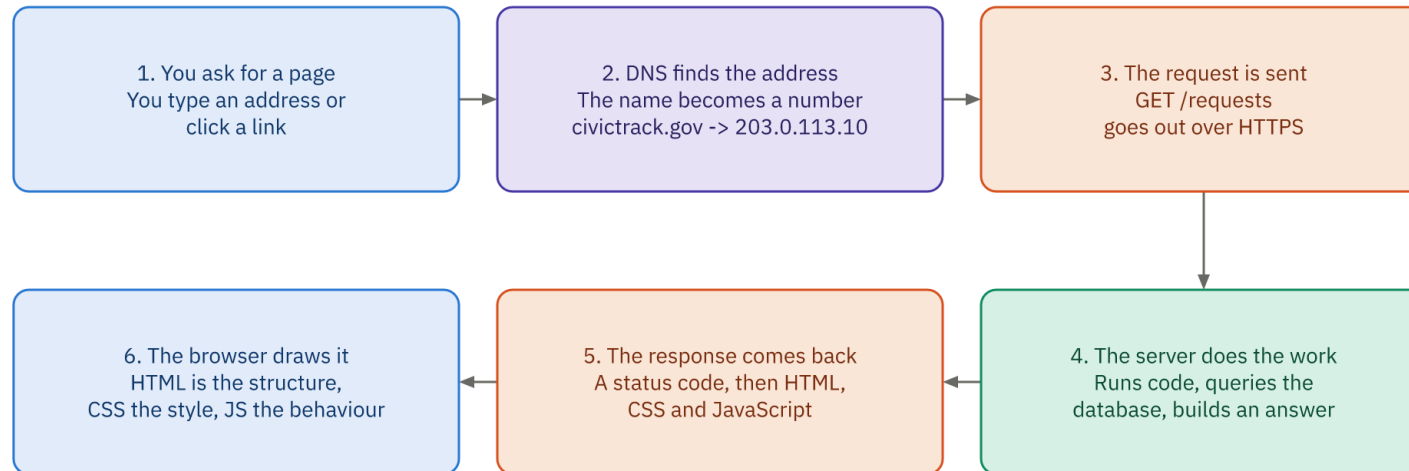
Client-Server Architecture

- Browser (client) sends a request
- Server processes and returns a response
- Request-response underlies the whole web
- Status codes: 200 OK, 404, 500
- HTTPS encrypts sensitive data in transit

What Happens When You Load a Page

What Happens When You Load a Page

Six steps, most of them finished before you notice the page was loading.



The status code in step 5 tells you which side has the problem:
200 OK · 404 the page does not exist · 500 the server broke

Steps 1, 2, 3 and 6 happen on the user's machine. Steps 4 and 5 happen somewhere you may never see.

Front-End: HTML, CSS, JavaScript

- HTML gives structure and content
- CSS controls style and layout
- JavaScript adds interactivity and behavior
- The DOM connects JavaScript to the page

```
let button = document.querySelector("button");
button.addEventListener("click", () => {
  alert("Added to cart!");
});
```

HTML, CSS, And JS Together

- Three layers combine into one page
- HTML structures; CSS styles; JS reacts
- Example: sliders update a color box live
- The same page adapts with responsive design
- Media queries adjust to screen size

APIs And The Full-Stack Picture

- API: a contract between front and back end
- REST uses GET, POST, PUT, DELETE
- Server responds with JSON data
- `fetch` calls APIs from the browser

```
fetch('/api/products/42')  
  .then(res => res.json())  
  .then(data => console.log(data.name));
```

Introduction To Java

Why Java

- Strongly typed, compiled, enterprise-grade
- Powers banks, insurance, government systems
- "Write once, run anywhere" via the JVM
- Backward compatible for decades
- Strong job market at large enterprises

Compiling And Running Java

- `javac` compiles source to bytecode
- `java` runs bytecode on the JVM
- Bytecode is CPU-independent and portable
- The compiler catches many errors early

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, world!");  
    }  
}  
  
// javac HelloWorld.java -> java HelloWorld
```

Variables And Types

- Every variable needs a declared type
- Primitives: `int`, `double`, `boolean`, `char`
- `String` is an object with methods
- Types can't change once declared
- Casting converts between number types

```
int age = 25;  
double price = 19.99;  
boolean active = true;  
String name = "Alice";
```

Control Flow, Loops, And Arrays

- `if / else` and `switch` for decisions
- `for`, `while`, and `do-while` loops
- Enhanced `for` iterates arrays cleanly
- Arrays are fixed size
- `ArrayList` is a resizable list

```
int[] nums = {1, 2, 3, 4, 5};  
for (int n : nums) {  
    if (n % 2 == 0) System.out.println(n); // 2, 4  
}
```

Classes And Objects

- Everything lives inside a class
- The constructor initializes an object's fields
- `new` creates objects from a class
- `this` refers to the current object
- Private fields hide internal data

```
public class Person {  
    private String name;  
    public Person(String name) {  
        this.name = name;  
    }  
    public void greet() {  
        System.out.println("Hi, I'm " + name);  
    }  
}
```

JavaScript vs. Java

Aspect	JavaScript	Java
Typing	Dynamic	Static
Running	Interpreted	Compiled
Classes	Optional	Required
Semicolons	Optional	Required

- JavaScript infers types; Java declares them
- Java catches type errors before running
- Same core ideas, different ceremony

Wrap-Up

Key Takeaways

- Concepts transfer; mostly the syntax changes
- Syntax is grammar; semantics is meaning
- JavaScript: flexible, interpreted, browser-native
- Java: typed, compiled, enterprise-scale
- Debugging is reading error messages closely
- Choose the language that fits the problem

Discussion Questions

- When would you pick compiled over interpreted?
- How do `==` and `===` differ in JavaScript?
- Why does Java require types and classes?
- Which errors are hardest to debug, and why?
- Front-end or back-end: where do business rules belong?